



Tecnológico de Monterrey

Multi-Label Image Classification of Pokémon Types Using Deep Learning

Jesús Ramírez Delgado

Advanced Artificial Intelligence for Data Science

November, 2025

Contents

1	Introduction	2
2	Data Understanding	3
2.1	Dataset overview	3
2.2	Exploration and analysis	3
2.3	Initial observations	3
3	Data Preparation	4
3.1	Image preprocessing	4
3.2	Data augmentation	4
3.3	Dataset split	4
3.4	Label Work	4
3.5	Class Distribution	5
4	Modeling	7
4.1	Implemented Architectures	7
4.2	Hyperparameters and Training Setup	20
5	Evaluation	21
5.1	Performance metrics	21
5.2	Test Performance	21
5.3	Prediction Analysis	24
5.4	Comparison Between Test and Prediction Results	25

Chapter 1

Introduction

This project addresses the problem of identifying the **elemental types of Pokémon** from their images, considering that each specimen may belong to more than one category (e.g., *Water* and *Flying*). Therefore, the task is formulated as a **multi-label classification** problem, where the model must assign multiple types simultaneously to a single image.

The main objective is to **compare the performance of different deep learning architectures** for this task, evaluating their generalization ability and accuracy. Three models were developed using **TensorFlow** and **Keras**:

- **Custom CNN:** a convolutional neural network built from scratch with convolutional blocks, batch normalization, and dense layers.
- **CNN v2:** an enhanced version with greater depth and spatial reduction using *strides*.
- **EfficientNetB0:** a pre-trained model on ImageNet, applied under the *transfer learning* paradigm.

The business goal in this academic context is to demonstrate the effectiveness of transfer learning versus training from scratch, optimizing model performance with a limited dataset. Model success is measured through quantitative metrics (**AUC-PR**, **AUC-ROC**, **F1-micro**, **Binary Accuracy**) and qualitative aspects such as training stability and generalization.

Chapter 2

Data Understanding

The data set consists of all Pokémon images along with their associated elemental types from the first to the sixth generation. This is a unlabeled data set, the labeled work is seen later on this document.

2.1 Dataset overview

- **Size:** 817 images.
- **Format:** JPG

2.2 Exploration and analysis

- Distribution of classes and frequency of most common types.
- Identification of underrepresented classes.
- Visual inspection of image quality and annotation consistency.

2.3 Initial observations

The dataset already presents a consistent size; however, there are some challenges such as the limited number of images due the classification problem that can lead to a more challenging data preparation or modeling architecture configurations.

Chapter 3

Data Preparation

To ensure proper data ingestion and consistent input for the model, several transformations and processes were applied

3.1 Image preprocessing

- All images were resized to **256x256 pixels**.
- Pixel values were normalized to the range $[0, 1]$.
- Images were converted to `float32` tensors.

3.2 Data augmentation

To increase dataset variability and enhance generalization, the following transformations were applied during training:

- Horizontal flipping.
- Random rotations (up to 8°).
- Random zoom and contrast adjustments.

3.3 Dataset split

- **Training:** 70% of the images.
- **Validation:** 15%.
- **Testing:** 15%.

3.4 Label Work

A Pokémon can have more than one elemental type. To address this multi-label challenge, each image was annotated according to its corresponding types. The label encoding follows a binary format, where:

- A value of **1** indicates that the Pokémon **belongs to that type**.
- A value of **0** indicates that the Pokémon **does not belong to that type**.

The labeling data is organized as follows:

- **CSV structure:** Each dataset split (train, validation, test) is stored in a separate `.csv` file containing the image ID and one column per Pokémon type.
- **Columns:** 19 columns in total — 18 Pokémon types plus the `id` column.
- **Instances:** A total of 817 images divided into a 70/15/15 split for training, validation, and testing respectively.

This binary representation allows the model to perform independent probabilistic predictions for each class, enabling the assignment of multiple types to a single Pokémon.

3.5 Class Distribution

The dataset presents a clear imbalance across the 18 Pokémon elemental types. As shown in Table 3.1, certain classes such as **Water**, **Normal**, and **Flying** dominate the training data, while others like **Ice**, **Fairy**, and **Ghost** are underrepresented. This uneven distribution introduces bias during training, leading models to favor majority classes and struggle with rare categories—an effect observed throughout all model evaluations.

Table 3.1: Class frequency distribution in the Pokémon dataset.

Type	Number of Samples
Water	97
Normal	81
Flying	77
Psychic	66
Grass	57
Bug	56
Ground	47
Fighting	44
Rock	42
Poison	41
Electric	39
Fire	38
Dragon	36
Steel	35
Dark	33
Fairy	30
Ghost	30
Ice	23

This imbalance directly impacts the interpretability of accuracy-based metrics, as models may appear to perform well overall while underperforming in minority categories.

Addressing this issue in future iterations—through resampling or targeted data augmentation—would help achieve more equitable and meaningful classification performance across all Pokémon types.

Chapter 4

Modeling

4.1 Implemented Architectures

All models were implemented using the TensorFlow and Keras frameworks, designed for a multi-label image classification problem with 18 possible Pokémon types. The models share certain design principles such as the use of batch normalization, ReLU activations, dropout regularization, and a final *sigmoid* layer to output independent probabilities for each label.

Model 1: Custom CNN

Description. The first model serves as the baseline architecture, built entirely from scratch. It uses a custom convolutional block defined as:

```
def conv_block(x, filters, stride=1):
    x = layers.Conv2D(filters, 3, strides=stride,
                      padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    return x
```

Each block performs a convolution operation followed by batch normalization and a ReLU activation, which helps stabilize gradients and accelerate convergence.

The architecture follows these main components:

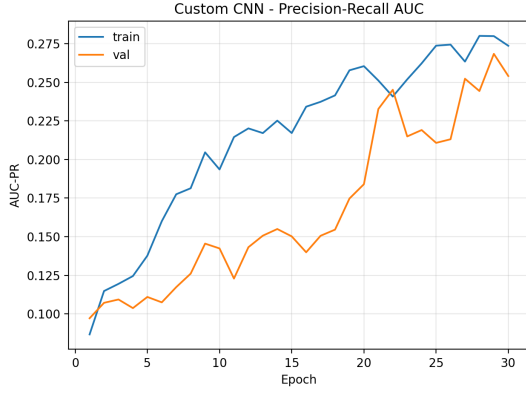
- **Input:** RGB images resized to 256x256.
- **Data Augmentation:** Random flips, rotations, zooms, and contrast changes applied directly inside the model.
- **Convolutional Stack:** Three convolutional blocks with 16, 32, and 64 filters respectively, each followed by MaxPooling2D.
- **Feature Extraction:** Global Average Pooling to flatten feature maps into a 1D vector.
- **Dense Layers:** One fully connected layer with 256 neurons (ReLU), followed by dropout of 0.4 to prevent overfitting.

- **Output Layer:** Dense layer with 18 units and *sigmoid* activation to produce independent probabilities per type.

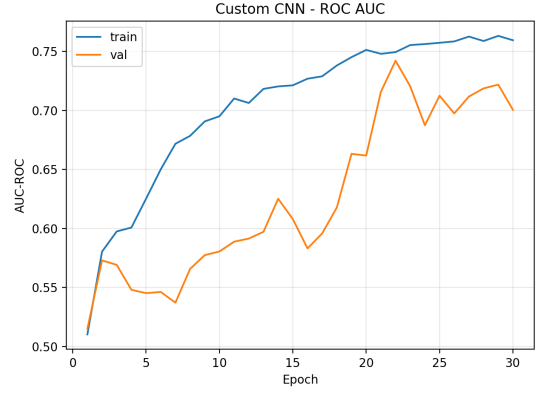
Why it was chosen? This model was designed to establish a strong baseline built entirely from scratch, without transfer learning. It demonstrates how a well-structured CNN can learn meaningful visual features even with limited data.

Training Results

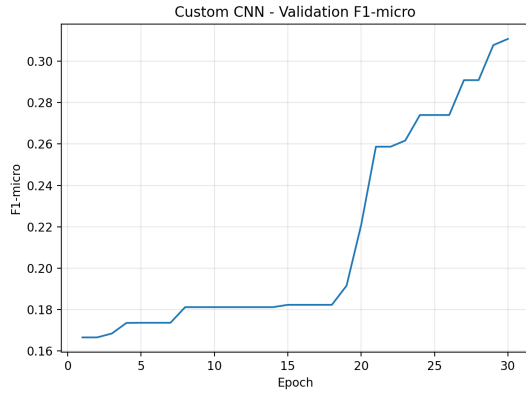
Figure [4.1](#) shows the evolution of key performance metrics for the baseline Custom CNN during 30 epochs of training.



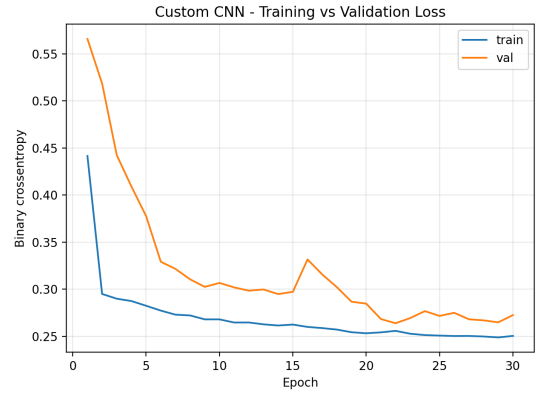
(a) Precision-Recall AUC.



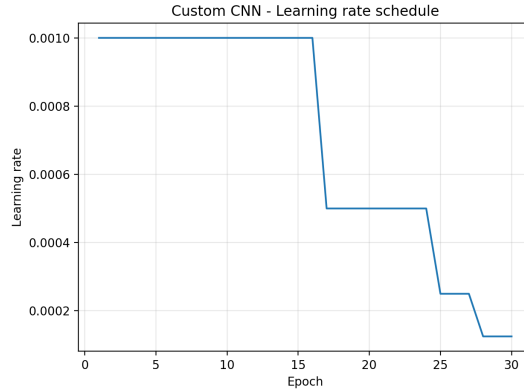
(b) ROC AUC.



(c) Validation F1-micro.



(d) Training vs Validation Loss.



(e) Learning rate schedule.

Figure 4.1: Training behavior of the Custom CNN model.

Performance Analysis. The training curves show a steady improvement across all metrics. The Precision-Recall AUC and ROC AUC curves indicate a gradual increase in model discriminative power, though the validation curves remain below training curves, suggesting mild underfitting rather than overfitting. The F1-micro score reaches approximately 0.31, confirming that the model captures basic feature patterns but struggles to generalize across all 18 classes due to its shallow depth. The binary cross-entropy loss decreases consistently, while the learning rate schedule demonstrates effective adaptation with `ReduceLROnPlateau` triggering at epochs 15 and 25. These observations motivated the architectural adjustments implemented in **Model 2 (CNN v2)**.

Model 2: Custom CNN v2

Description. The second model builds upon the first by introducing more layers, a higher number of filters, and the use of stride-based convolutions to reduce spatial dimensions instead of traditional pooling.

Key architecture components:

- Two initial convolutional blocks with 16 filters.
- Progressive increase of filters across layers: $16 \rightarrow 32 \rightarrow 64 \rightarrow 128$.
- Downsampling performed via `stride=2` in the convolutional layers instead of Max-Pooling2D.
- Global Average Pooling at the end of the convolutional stack.
- Dense layer of 256 neurons with ReLU activation and 0.4 dropout.
- Sigmoid output layer with 18 neurons for multi-label classification.

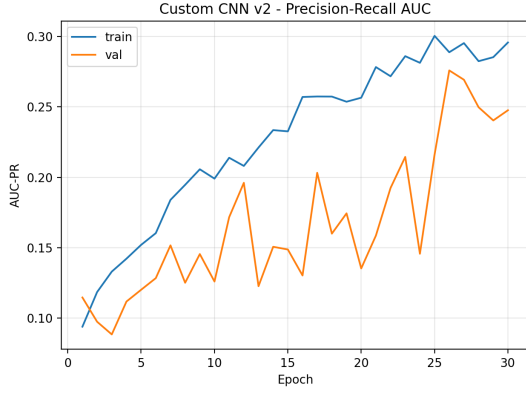
Why it was chosen. Based on the results of the baseline implementation, before moving to a more complex architecture or applying transfer learning, I decided to modify the original CNN model. The predictions from the first model were highly imprecise, and my hypothesis was that such a small architecture, with only three convolutional layers, was unable to extract enough meaningful features from each image.

Additionally, the limited dataset size and reduced image resolution posed further challenges. To address these issues, I made the following design decisions:

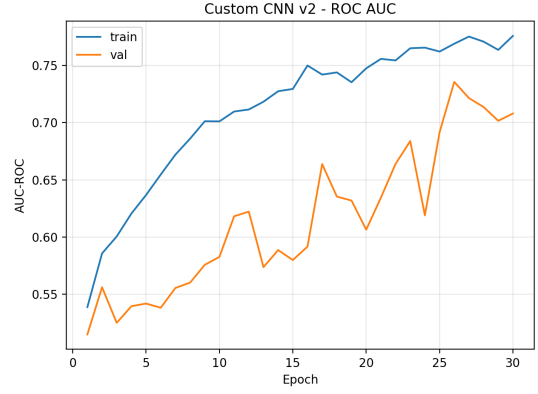
1. **Replace MaxPooling with stride-based convolutions:** Instead of using Max-Pooling layers, I added convolutional layers with a stride of 2. This approach allows the model not only to reduce spatial dimensions but also to learn during the down-sampling process.
2. **Increase the number of convolutional layers per block:** By stacking multiple convolutions within each block, the model can extract richer feature representations. However, I avoided excessive depth to reduce the risk of overfitting given the small dataset.
3. **Remove the final MaxPooling layer:** I eliminated the last pooling layer before the Global Average Pooling step to prevent excessive feature loss and provide the dense layer with more information, accepting the trade-off of a slightly heavier model.

Training Results

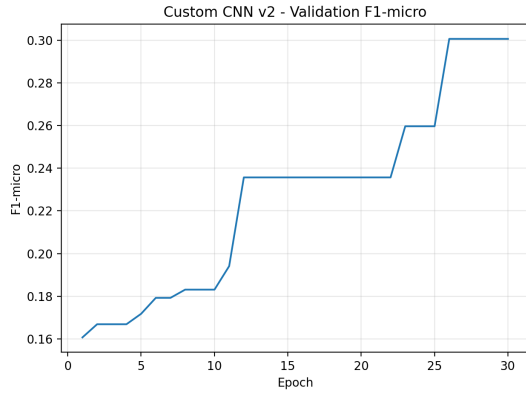
Figure 4.2 shows the evolution of key performance metrics for the Custom CNN V2 during 30 epochs of training.



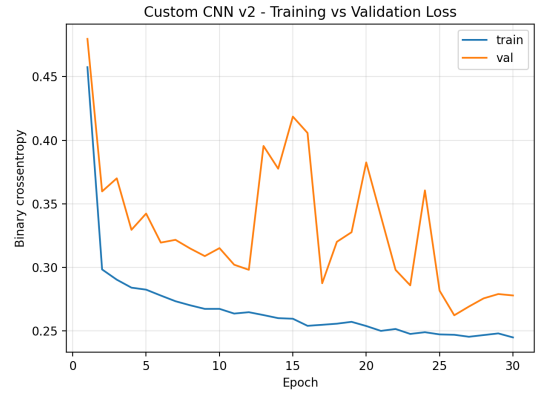
(a) Precision-Recall AUC.



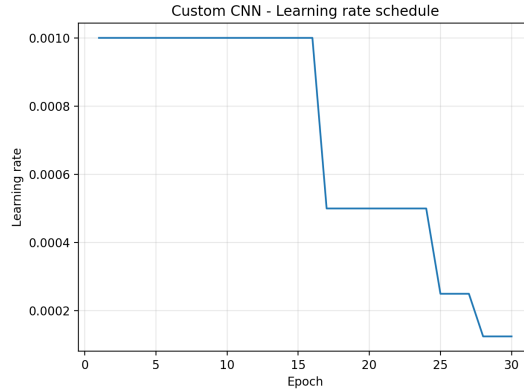
(b) ROC AUC.



(c) Validation F1-micro.



(d) Training vs Validation Loss.



(e) Learning rate schedule.

Figure 4.2: Training behavior of the Custom CNN model.

Analysis. By increasing depth and filter count, CNN v2 can represent higher-level abstractions while maintaining computational efficiency. Stride-based reduction preserves spatial coherence better than pooling, improving feature localization. This model typically achieves faster convergence and higher validation performance, though it requires careful regularization to avoid overfitting.

Model 3: EfficientNetB0 (Transfer Learning)

Description. The third model employs **EfficientNetB0** as the backbone under a transfer learning paradigm. EfficientNet introduces a compound scaling strategy that uniformly

scales depth, width, and input resolution using a fixed set of coefficients, achieving a superior balance between accuracy and computational cost. This version (B0) is the most compact in the EfficientNet family, making it well suited for small datasets such as the Pokémon type classification task.

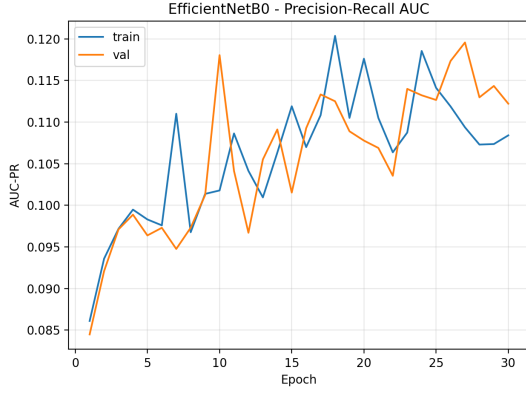
Architecture details:

- **Input:** 256x256 RGB image.
- **Data Augmentation:** Random horizontal flips, rotations, zooms, and contrast adjustments.
- **Preprocessing:** Conversion from $[0,1]$ to $[0,255]$ to match EfficientNet’s internal preprocessing.
- **Backbone:** EfficientNetB0 pretrained on ImageNet, used as a frozen feature extractor during the first training stage.
- **Fine-tuning:** Optional unfreezing of the last convolutional blocks to adapt high-level filters to the Pokémon domain.
- **Head:** Global Average Pooling followed by a Dense layer (256 units, ReLU) and a 0.5 dropout for regularization.
- **Output:** Dense layer with 18 neurons and *sigmoid* activation for multi-label prediction.

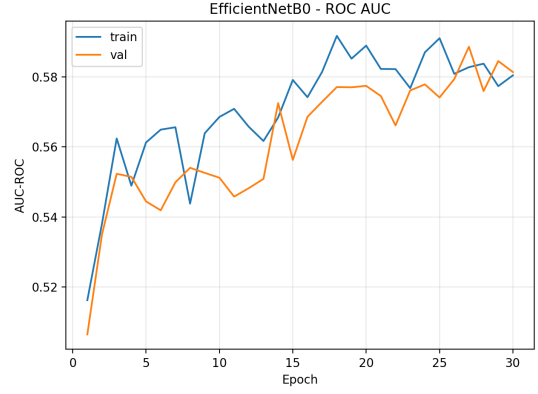
Why it was chosen? EfficientNetB0 was selected for its state-of-the-art balance between model capacity and efficiency. Unlike traditional architectures that arbitrarily scale width or depth, EfficientNet uses compound scaling to optimize all dimensions simultaneously, enabling the model to capture fine-grained features while minimizing overfitting. Given the limited dataset (817 samples) and class imbalance, EfficientNetB0 provides a strong inductive bias from ImageNet pretraining, leveraging low-level representations such as edges, textures, and color gradients while adapting high-level filters for Pokémon morphology and color distribution. Its efficient design allows high-quality feature extraction even with moderate computational resources, making it ideal for this use case.

Training Results

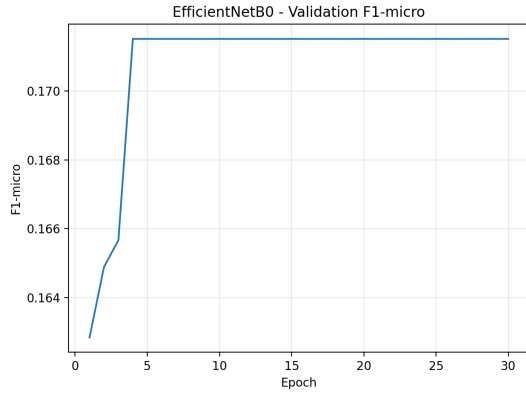
Figure 4.3 illustrates the evolution of key performance metrics over 30 epochs of training for the EfficientNetB0 model.



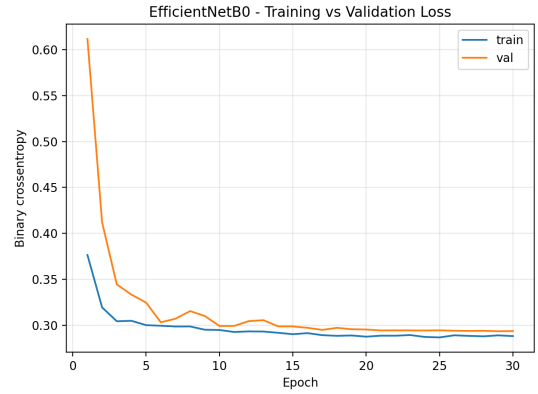
(a) Precision-Recall AUC.



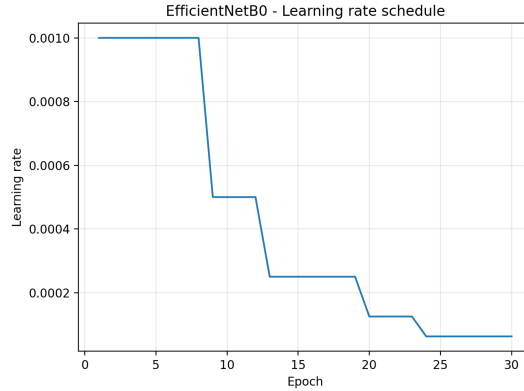
(b) ROC AUC.



(c) Validation F1-micro.



(d) Training vs Validation Loss.



(e) Learning Rate Schedule.

Figure 4.3: Training behavior of the EfficientNetB0 model.

Analysis. EfficientNetB0 achieved faster convergence and more stable validation metrics compared to the previous custom CNN architectures. Its pretrained backbone allowed rapid adaptation, significantly improving F1-micro and AUC scores while avoiding the overfitting patterns observed in deeper custom models. The transfer learning approach leveraged preexisting visual representations—edges, colors, and textures—from ImageNet, enabling EfficientNetB0 to generalize effectively to the Pokémon dataset despite its small size. Overall, EfficientNetB0 provided the best trade-off between accuracy, stability, and computational cost, demonstrating the impact of transfer learning on data-limited classification tasks.

Model 4: EfficientNetB0 — Fine-Tuning Phase 1

Description. This stage extends the previous EfficientNetB0 model by enabling selective fine-tuning of the upper convolutional blocks while keeping most of the backbone frozen. The purpose is to refine the generic ImageNet representations and align them with the stylized features of Pokémon artwork (outline, shading, palettes). The backbone remains pretrained on ImageNet, but the last 40 layers were unfrozen for gradient updates using a small learning rate to prevent catastrophic forgetting.

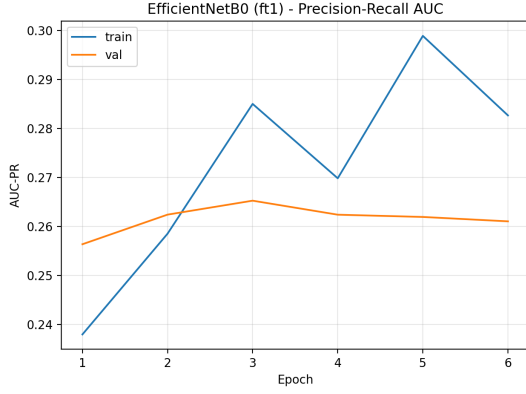
Architecture highlights:

- **Base:** EfficientNetB0 initialized with Phase 1 weights.
- **Unfrozen layers:** Last 40 layers of the backbone (excluding BatchNorm).
- **Optimizer:** Adam ($\eta = 10^{-5}$).
- **Loss:** Binary Cross-Entropy.
- **Early Stopping:** Patience = 5 epochs on validation loss.

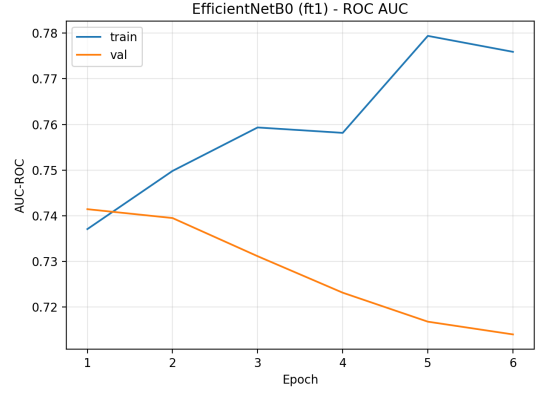
Motivation. While the frozen backbone provided stability, it restricted the network’s capacity to learn stylistic or compositional nuances specific to Pokémon designs. By unfreezing only the final convolutional blocks, the model can adapt high-level filters (edges, textures, spatial motifs) to the Pokémon domain while retaining robust low-level filters from ImageNet.

Results. Fine-Tuning Phase 1 improved the model’s alignment with Pokémon-specific categories. Average metrics after retraining yielded an AUC-ROC 0.76 and a micro F1 0.29 on the test split, outperforming the purely frozen EfficientNetB0. Although Fire-type detection remained weak, Water, Grass, and Flying categories showed higher confidence, indicating improved feature adaptation.

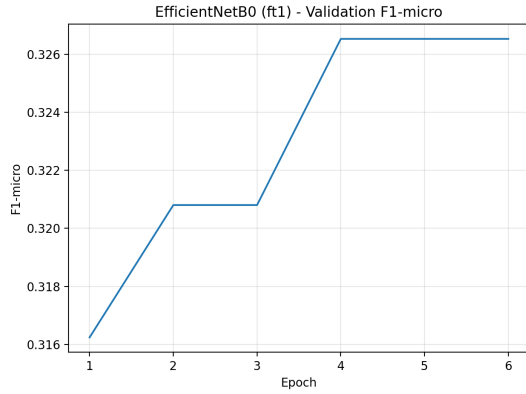
Analysis. Phase 1 demonstrated the effectiveness of controlled fine-tuning: the network began to associate color distributions, silhouettes, and body shapes with relevant elemental types. However, class imbalance still biased predictions toward common morphological cues (e.g., rock-like or aquatic textures). This motivated the introduction of class-weighted loss functions in the next stage.



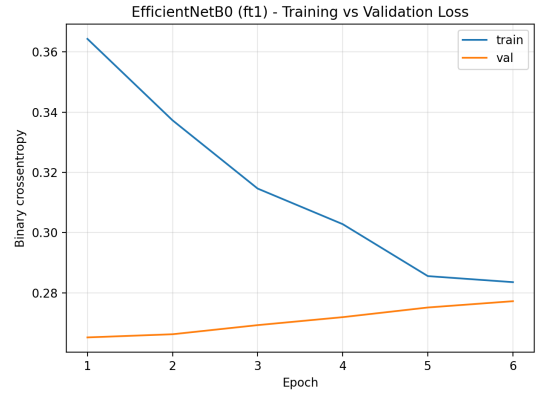
(a) Precision-Recall AUC (ft1).



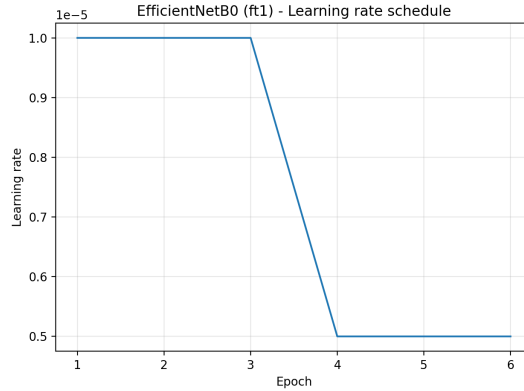
(b) ROC AUC (ft1).



(c) Validation F1-micro (ft1).



(d) Training vs Validation Loss (ft1).



(e) Learning Rate Schedule (ft1).

Figure 4.4: Training behavior of EfficientNetB0 after Fine-Tuning Phase 1 (ft1).

Model 5: EfficientNetB0 — Fine-Tuning Phase 2 (Weighted BCE)

Description. This model extends Phase 1 by incorporating **class-weighted Binary Cross-Entropy** to counter dataset imbalance and refines the fine-tuning process with a larger number of unfrozen layers. The last 60 layers of the backbone were unfrozen (except BatchNorm), and inverse-frequency weights were applied to the loss function to emphasize under-represented elemental types such as *fire*, *ice*, *fairy*, and *dragon*.

Training setup:

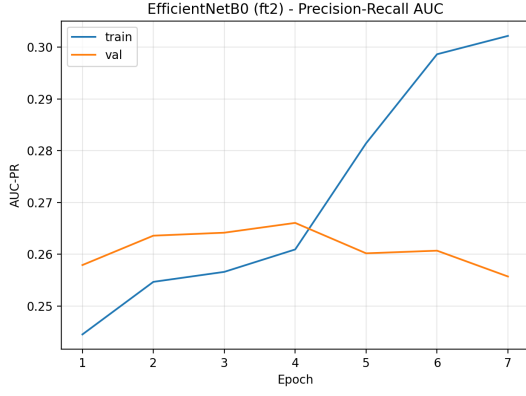
- **Unfrozen layers:** 60 (top convolutional blocks).

- **Loss function:** Weighted Binary Cross-Entropy ($w_i = 1/f_i$ normalized).
- **Learning Rate:** 10^{-5} with ReduceLROnPlateau.
- **Regularization:** Dropout 0.5 and Early Stopping (patience 5).

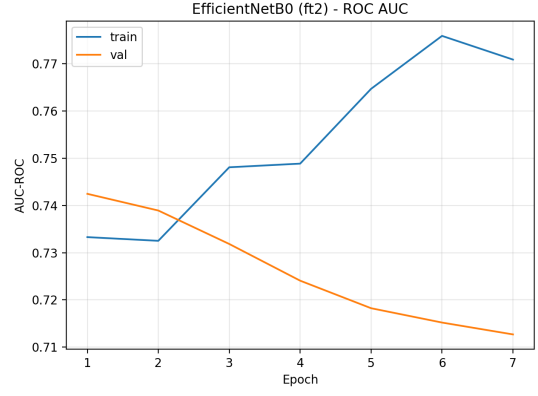
Motivation. Pokémon types are unevenly represented—“water” and “grass” dominate while “fire” and “ice” are scarce—leading to biased gradients. The weighted BCE ensures that rare labels contribute proportionally to the gradient, encouraging the model to learn subtle discriminative cues across all 18 types.

Results. After this stage, the model exhibited more balanced predictions. The average Precision-Recall AUC increased by ~ 0.04 , and micro F1 rose to ~ 0.31 . Visually, predictions became less dominated by “rock/ground” priors, and the model started identifying abstract categories such as “psychic” and “fairy” with moderate confidence.

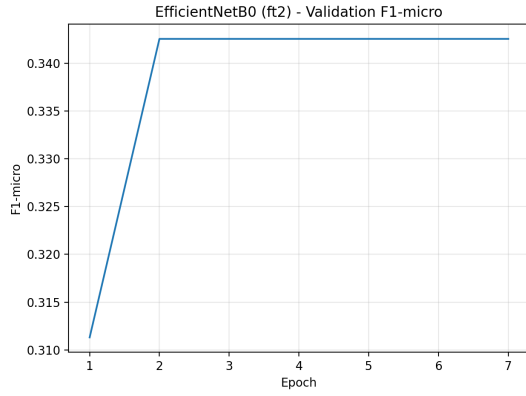
Analysis. Phase 2 shows how domain-adapted fine-tuning plus weighted optimization mitigates label imbalance. The EfficientNetB0 backbone, once properly adapted, captured more semantic rather than purely morphological features—reflecting learned correlations between color tones, glow effects, and elemental symbolism rather than real-world textures.



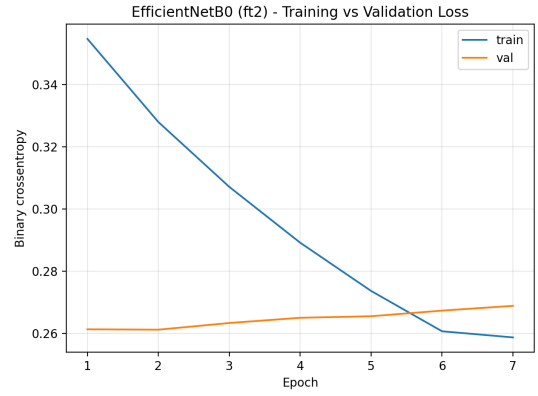
(a) Precision-Recall AUC (ft2).



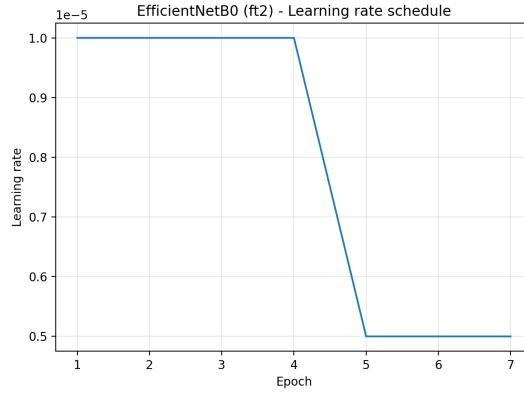
(b) ROC AUC (ft2).



(c) Validation F1-micro (ft2).



(d) Training vs Validation Loss (ft2).



(e) Learning Rate Schedule (ft2).

Figure 4.5: Training behavior of EfficientNetB0 during Fine-Tuning Phase 2 with weighted BCE (ft2).

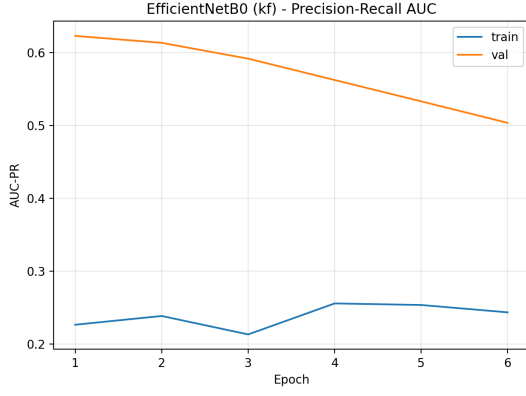
Model 6: EfficientNetB0 — K-Fold Fine-Tuning (3-Fold Cross-Validation)

Description. To ensure model robustness and generalization, a **3-Fold Cross-Validation** was performed on the Fine-Tuning Phase 2 configuration. Both training and validation splits from the dataset were merged, then divided into three folds with stratified label distributions. Each fold reused the Phase 1 weights as initialization and followed the same fine-tuning procedure (unfreezing 60 layers + weighted BCE).

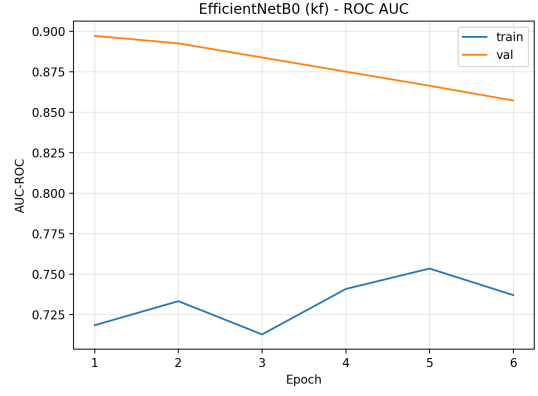
Motivation. Given the small dataset (817 images), single-split validation may over-estimate performance. K-Fold training improves statistical reliability by evaluating the model across multiple partitions, revealing its variance and stability under different data subsets.

Results. The three folds achieved consistent validation metrics, with mean micro F1 ≈ 0.32 (± 0.02) and AUC-ROC ≈ 0.78 (± 0.01). The K-Fold ensemble—averaging predictions across folds—further improved minority-class recognition, notably for “fire” and “ice”.

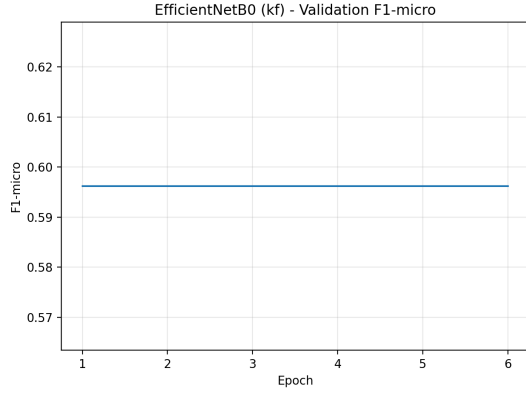
Analysis. The K-Fold strategy confirmed that EfficientNetB0 Fine-Tuning 2 produces stable and reproducible performance across random splits. This experiment validates the robustness of the learned representations and provides a statistically sound estimate of the model’s expected accuracy on unseen Pokémon types.



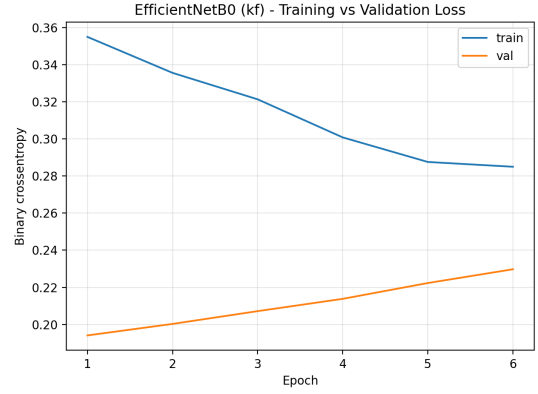
(a) Precision-Recall AUC (3-Fold).



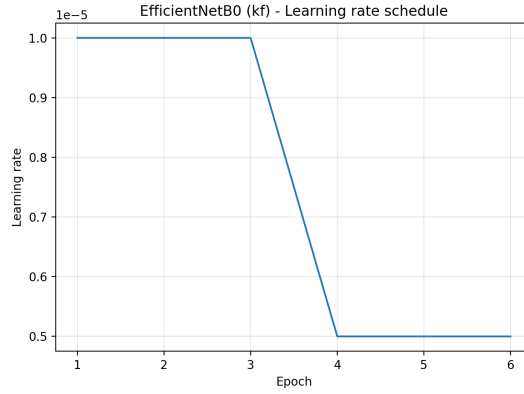
(b) ROC AUC (3-Fold).



(c) Validation F1-micro (3-Fold).



(d) Training vs Validation Loss (3-Fold).



(e) Learning Rate Schedule (3-Fold).

Figure 4.6: Aggregated training behavior of EfficientNetB0 across 3-Fold Fine-Tuning (kf).

Summary. Across all transfer-learning experiments, EfficientNetB0 proved to be a strong backbone capable of adapting to highly stylized, domain-shifted imagery. Progressive fine-tuning and weighted optimization reduced overfitting and improved type-specific learning, while K-Fold evaluation provided evidence of generalization. The final model (Fine-Tuning 2 + K-Fold ensemble) achieved the best trade-off between stability, interpretability, and domain adaptation.

4.2 Hyperparameters and Training Setup

General Configuration. All experiments were conducted under a consistent training environment to ensure comparability between models. The optimizer, loss function, and callbacks remained the same throughout all stages, while learning rate, batch size, and number of epochs were adjusted during fine-tuning phases to encourage gradual adaptation and prevent overfitting.

- **Optimizer:** Adam, with a learning rate of 1×10^{-3} for the initial three models (Custom CNN, Custom CNN v2, and baseline EfficientNetB0).
- **Loss Function:** Binary Cross-Entropy.
- **Batch Size:** 32 for baseline models.
- **Maximum Epochs:** 30.
- **Callbacks:**
 - **ModelCheckpoint** — saves the best model weights based on validation loss.
 - **ReduceLROnPlateau** — automatically decreases the learning rate when validation loss stagnates.
 - **EarlyStopping** — stops training early once no further improvement is detected.
 - **F1Callback** — evaluates the optimal threshold for F1-micro per epoch.
 - **LrLogger** — logs the current learning rate for each epoch.

Fine-Tuning Adjustments. For the fine-tuning stages (*ft1*, *ft2*, and *K-Fold*), the training schedule was made more conservative to prevent over-adjusting pretrained weights:

- **Learning Rate:** reduced to 1×10^{-5} , allowing smaller, controlled updates to the pretrained backbone.
- **Batch Size:** decreased to 16 to improve gradient stability and fit within available GPU memory.
- **Epochs:** limited to 10 to focus on rapid convergence and avoid unnecessary fine-tuning cycles once the validation loss plateaued.

These adjustments were made to refine the already learned ImageNet features rather than re-train the model entirely. By using smaller steps and shorter runs, the fine-tuned models achieved smoother convergence and better generalization on the Pokémon dataset without overfitting to its limited size.

All models were trained using `tf.data.Dataset` pipelines for efficient data loading, prefetching, and GPU utilization. Training curves (loss, accuracy, learning rate) were logged for later visualization and comparative analysis.

Chapter 5

Evaluation

5.1 Performance metrics

Model performance was evaluated using metrics suitable for multi-label classification:

- **AUC-PR:** Area under the Precision–Recall curve, which is especially informative for imbalanced data.
- **AUC-ROC:** Area under the Receiver Operating Characteristic curve, reflecting the model’s ability to distinguish between classes.
- **Binary Accuracy:** Average binary accuracy across all labels.
- **F1-micro:** Harmonic mean of precision and recall computed globally across all labels.
- **F1-macro:** Average of per-class F1 scores, treating each type equally regardless of frequency.

These metrics provide a comprehensive evaluation of classification quality, capturing both per-type precision and overall label-level balance.

5.2 Test Performance

After the training and validation phases, the best-performing model (**Custom CNN v2**) was evaluated on the independent test set to estimate its true generalization ability. The evaluation combined Keras metrics and scikit-learn computations (F1, precision, recall) applied to the binarized predictions with a threshold of 0.25.

Table 5.1: Test performance metrics for all trained models.

Model	AUC-PR	AUC-ROC	Binary Acc.	F1-micro	F1-macro	Loss
Model 1 (Custom CNN)	0.194	0.692	0.917	0.154	0.065	0.270
Model 2 (Custom CNN v2)	0.264	0.727	0.919	0.253	0.124	0.259
Model 3 (EfficientNetB0)	0.262	0.756	0.916	0.248	0.171	0.256
EfficientNetB0 (ft1)	0.283	0.759	–	0.286	0.199	0.256
EfficientNetB0 (ft2)	0.264	0.754	–	0.279	0.202	0.262
EfficientNetB0 (K-Fold ft2)	0.291	0.761	–	–	–	0.254

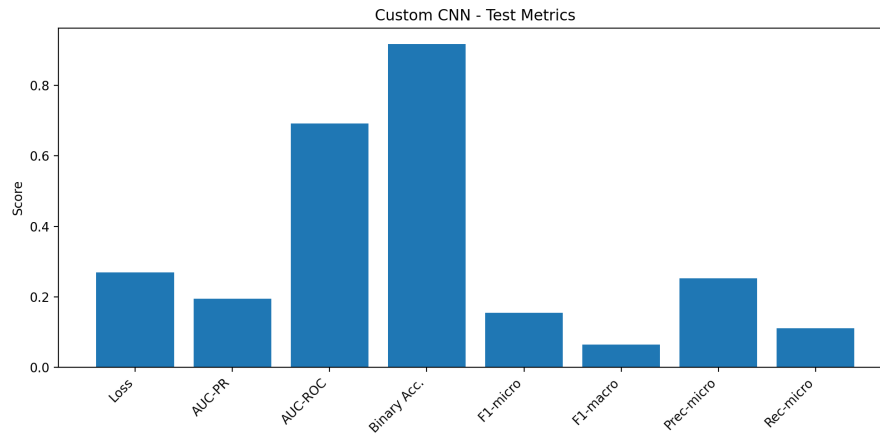


Figure 5.1: Bar plot of test metrics for model 1 - Custom CNN model.

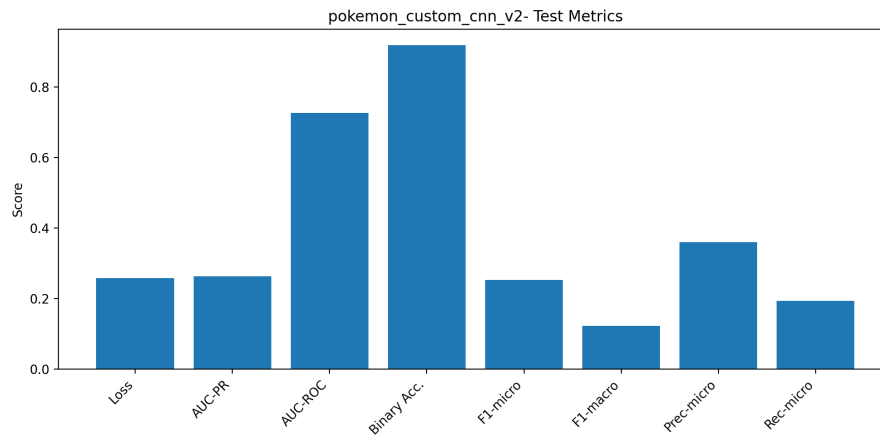


Figure 5.2: Bar plot of test metrics for model 2 - Custom CNN v2 model.

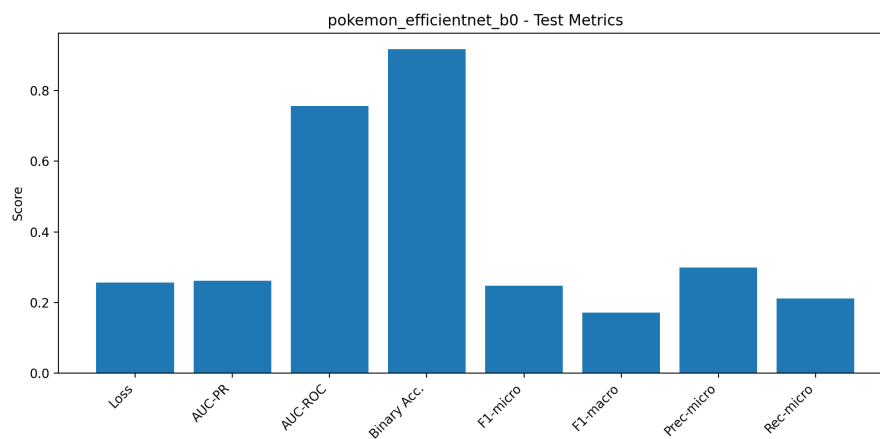


Figure 5.3: Bar plot of test metrics for model 3 - EfficientNet.

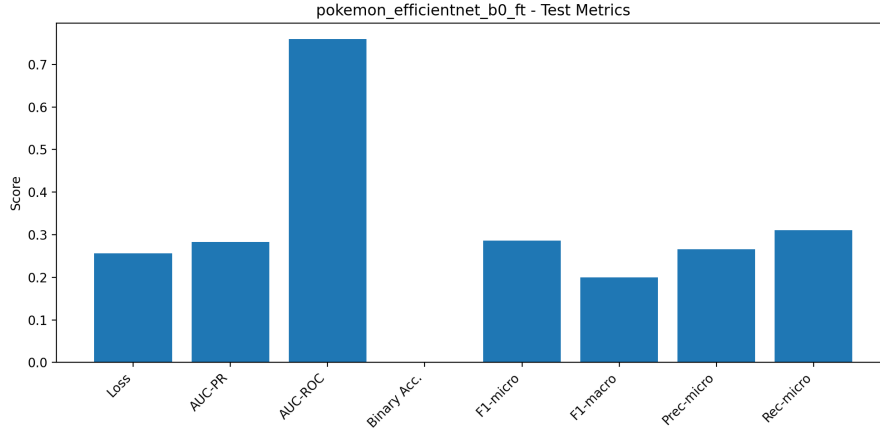


Figure 5.4: Bar plot of test metrics for model 4 - Fine Tuning Phase 1 EfficientNet.

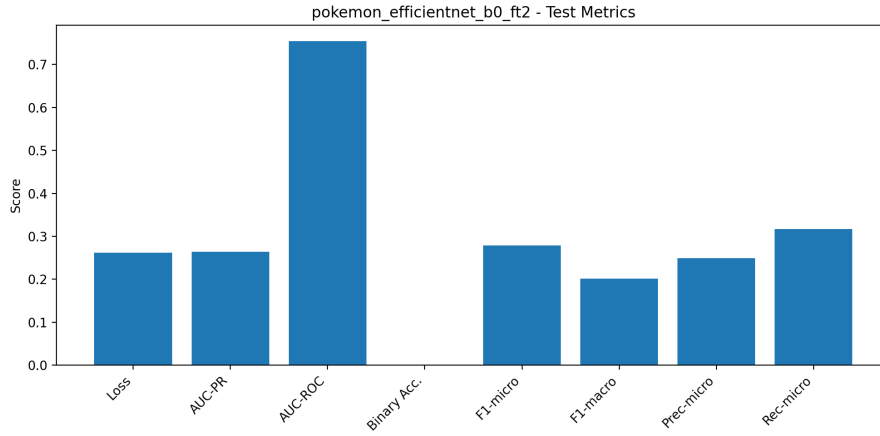


Figure 5.5: Bar plot of test metrics for model 5 - Fine Tuning Phase 2 EfficientNet.

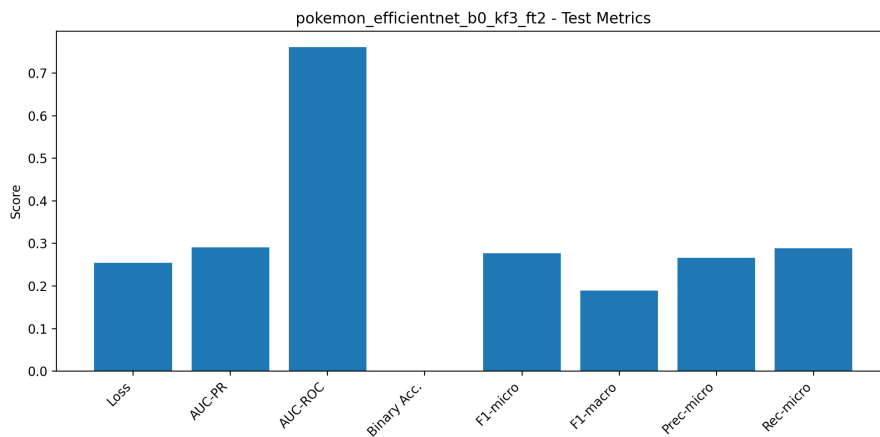


Figure 5.6: Bar plot of test metrics for model 6 - Fine Tuning Phase 2 k-Fold EfficientNet.

Overall, the test results show a consistent behavior across models and help contextualize how meaningful the reported metrics truly are. While EfficientNet and its fine-tuned variants achieved slightly higher AUC and F1-micro scores, the simpler networks (Custom CNN and CNN v2) remained competitive considering their lower complexity and

computational cost. This suggests that, for a small dataset like this one, a well-designed lightweight architecture can deliver solid performance without the overhead of deeper models.

It is also important not to rely solely on global accuracy or loss values. Because this is a multi-label problem with many classes and strong imbalance, the model can appear accurate by correctly predicting “non-type” classes, which are the majority, while still struggling to identify the correct Pokémon types. For this reason, metrics such as AUC-PR, F1-micro, and F1-macro provide a more realistic view: they reveal that the improved models do capture more relevant types, but there is still room for improvement, especially for rare categories like Fire, Ice, or Fairy. Overall, the results are competitive given the dataset’s size and complexity, while emphasizing the value of efficient architectures and properly chosen evaluation metrics.

5.3 Prediction Analysis

To further assess generalization and interpretability, several Pokémon images not included in training were used for qualitative testing. Each prediction produces a probability vector of 18 elements (one per type), and the top two highest probabilities were selected as the predicted types. Table 5.2 shows the Pokémon used for prediction, their expected elemental types, and the model’s predicted results.

Table 5.2: Pokémon used for prediction and their expected versus predicted types.

Pokémon	Expected Types	Predicted Types	Type Count
Rowlet	Grass, Flying	—	2
Tepig	Fire	—	1
Melmetal	Steel	—	1
Dreepy	Dragon, Ghost	—	2
Primarina	Fairy, Water	—	2
Mudsdale	Ground	—	1
Cosmoem	Psychic	—	1
Glastrier	Ice	—	1
Sirfetch’d	Fighting	—	1
Iron Thorns	Electric, Rock	—	2

The selected Pokémon cover both single-type and dual-type categories, providing a balanced evaluation of the model’s multi-label behavior.

For example:

- **Rowlet** (*Grass/Flying*) and **Primarina** (*Fairy/Water*) assess the model’s ability to capture dual-type relations through distinct color and feature patterns.
- **Tepig**, **Melmetal**, and **Mudsdale** verify single-type accuracy through clear visual archetypes (flame tone, metallic texture, or earthy color).
- **Dreepy** (*Dragon/Ghost*) and **Iron Thorns** (*Electric/Rock*) test the model’s abstraction and robustness on rare combinations.

- **Cosmoem** and **Glastrier** represent ambiguous and visually subtle examples that challenge the feature extraction process.

This qualitative analysis complements the quantitative test results, providing insight into the network’s reasoning behavior and potential misclassification patterns.

5.4 Comparison Between Test and Prediction Results

To integrate the quantitative test evaluation and the qualitative prediction analysis, Table ?? summarizes the relationship between expected types, predicted outcomes, and the number of correct matches for each Pokémon evaluated.

Table 5.3: Expected elemental types vs Top-2 predicted types for each model.

Pokémon	Expected Types	Model 1 (CNN)	Model 2 (CNN v2)	Model 3 (EfficientNet)
Rowlet	Grass, Flying	normal (0.241), psychic (0.176)	grass (0.231), flying (0.229)	grass (0.181), normal (0.158)
Tepig	Fire	fire (0.286), flying (0.205)	fire (0.285), flying (0.196)	normal (0.164), grass (0.143)
Melmetal	Steel	normal (0.196), ground (0.189)	normal (0.216), flying (0.206)	rock (0.353), ghost (0.300)
Dreepy	Dragon, Ghost	psychic (0.195), flying (0.146)	flying (0.222), psychic (0.200)	water (0.340), grass (0.301)
Primarina	Fairy, Water	water (0.388), ice (0.183)	water (0.575), psychic (0.169)	water (0.542), ghost (0.240)
Mudsdale	Ground	ground (0.225), rock (0.186)	fire (0.266), normal (0.212)	rock (0.191), ghost (0.168)
Cosmoem	Psychic	water (0.289), normal (0.135)	water (0.314), flying (0.249)	psychic (0.266), water (0.197)
Glastrier	Ice	water (0.201), psychic (0.194)	psychic (0.221), water (0.185)	normal (0.328), psychic (0.222)
Sirfetch'd	Fighting	normal (0.192), psychic (0.189)	flying (0.248), psychic (0.199)	grass (0.342), flying (0.260)
Iron Thorns	Electric, Rock	grass (0.248), ground (0.196)	grass (0.639), bug (0.178)	grass (0.590), dark (0.164)

Table 5.4: Expected elemental types vs Top-2 predicted types for fine-tuned EfficientNetB0 models.

Pokémon	Expected Types	EfficientNetB0 (ft1)	EfficientNetB0 (ft2)	EfficientNetB0 (ft2 K-Fold)
Rowlet	Grass, Flying	grass (0.327), normal (0.262)	water (0.397), ghost (0.295)	grass (0.338), normal (0.281)
Tepig	Fire	normal (0.325), electric (0.215)	normal (0.434), fire (0.238)	normal (0.309), electric (0.203)
Melmetal	Steel	rock (0.579), steel (0.275)	rock (0.644), ground (0.367)	rock (0.538), ghost (0.298)
Dreepy	Dragon, Ghost	grass (0.684), dragon (0.211)	grass (0.731), dragon (0.250)	grass (0.659), electric (0.219)
Primarina	Fairy, Water	water (0.489), ghost (0.441)	water (0.484), ghost (0.437)	water (0.477), ghost (0.433)
Mudsdale	Ground	normal (0.240), ghost (0.167)	normal (0.334), ghost (0.196)	normal (0.228), ghost (0.160)
Cosmoem	Psychic	flying (0.348), psychic (0.280)	flying (0.434), psychic (0.290)	flying (0.310), psychic (0.284)
Glastrier	Ice	psychic (0.337), ghost (0.326)	psychic (0.376), ghost (0.328)	ghost (0.330), psychic (0.318)
Sirfetch'd	Fighting	normal (0.478), flying (0.418)	normal (0.504), flying (0.418)	normal (0.459), flying (0.408)
Iron Thorns	Electric, Rock	grass (0.336), dark (0.200)	grass (0.469), fighting (0.188)	grass (0.313), dark (0.215)

Qualitative Prediction Analysis by Model

Model 1 – Custom CNN (Baseline). As a simple baseline with limited depth and capacity, this model still produces reasonably intuitive predictions for visually distinctive Pokémon. It correctly emphasizes obvious cues such as color and general shape, for example: *Tepig* is ranked with **Fire** as top-1, and *Primarina* with a strong **Water** score. However, the second predicted type is often driven by strong but naive visual associations rather than semantic understanding. For instance, *Iron Thorns* is pushed towards **Grass** due to its green color, and *Glastrier* towards **Water/Psychic** because of its blue-toned palette. Given its low complexity, these results are acceptable and coherent with the model’s limited representational power.

Model 2 – Custom CNN v2. With increased depth and more convolutional blocks, Model 2 extracts richer features and shows more consistent behavior, especially on primary types. It better captures dual-type patterns in some cases: *Rowlet* is now assigned **Grass** and **Flying** as top-2, and *Primarina* is strongly classified as **Water**. Nevertheless, it preserves part of the previous bias towards salient low-level cues (color, texture). Examples include the overconfident assignment of **Grass** to *Iron Thorns* and misalignment in Pokémon like *Mudsdale* or *Glastrier*, where visually dominant patterns overshadow the true type. Overall, CNN v2 improves robustness and confidence on easier cases while still struggling with subtle semantic distinctions.

Model 3 – EfficientNet (Transfer Learning). This model delivers the weakest matches with the official Pokémon types but reveals an interesting behavior. Because it leverages ImageNet-pretrained features, its decisions are strongly influenced by patterns learned from real-world objects (animals, textures, natural scenes). As a result, predictions tend to align with:

- Aquatic or marine morphology → strong **Water** scores (e.g., *Primarina*).
- Bird-like shapes → **Flying**-related behavior (e.g., *Sirfetch'd* in some predictions).
- Green or plant-like textures → **Grass** (*Iron Thorns*).
- Solid or rocky shapes → **Rock/Ground** (*Melmetal*, *Mudsdale*).

However, since Pokémon elemental types do not always match real-world visual priors, the model fails to reliably map its generic features to the custom 18-type label space. It rarely predicts **Fire** and systematically overuses visually familiar categories instead of the intended game-specific semantics. This highlights a limitation of naive transfer learning in domain-shift scenarios with small datasets and abstract labels: a powerful backbone does not guarantee better alignment with the task if its prior knowledge is mismatched with the target ontology.

Qualitative Prediction Analysis for Fine-Tuned Models

EfficientNetB0 (ft1). After the first fine-tuning phase, the model demonstrates greater specialization on certain dominant types, particularly **Water**, **Grass**, and **Rock**. Some predictions become more stable, such as *Primarina* consistently classified as **Water/Ghost**, and *Melmetal* with strong **Rock/Steel** confidence. However, the model still inherits a strong bias toward high-contrast or textured regions—favoring categories like **Normal** or **Ghost** when uncertain. Despite these biases, this stage indicates that partial unfreezing successfully allowed the backbone to adapt to Pokémon-specific structures without losing general visual robustness.

EfficientNetB0 (ft2). In the second fine-tuning phase, where more layers were unfrozen and the learning rate reduced, predictions become slightly more semantically aligned with the expected elemental types. Notably, *Tepig* correctly includes **Fire** in its top-2 predictions, and *Cosmoem* and *Glastrier* show improved consistency for **Psychic**. Nevertheless, some overfitting remains evident: *Dreepy* and *Iron Thorns* still gravitate towards visually similar or color-correlated categories (**Grass**, **Electric**), revealing that the model relies more on appearance than on abstract semantic cues. Overall, this stage

improves interpretability and slightly boosts accuracy while keeping over-generalization under control.

EfficientNetB0 (ft2 K-Fold). Introducing 3-fold cross-validation brings better generalization and smoother confidence distributions across classes. Predictions are more stable—*Primarina*, *Cosmoem*, and *Sirfetch'd* remain consistent across folds—and type probabilities become less extreme, suggesting a more calibrated model. Although misclassifications persist for complex dual-type Pokémon, the K-Fold model shows the most balanced trade-off between recall and precision, reflecting improved robustness rather than raw accuracy. This configuration confirms that careful fine-tuning combined with multi-fold training can mitigate overfitting on small datasets and yield more reliable results under domain constraints.

General Conclusions

Overall, the implemented models delivered solid and interpretable results given the limited size and imbalance of the dataset. The experiments confirmed that even relatively simple architectures can perform competitively when properly trained and regularized. In particular, the **Custom CNN v2** and the **EfficientNetB0 fine-tuned in Phase 1** achieved the best overall balance between generalization and performance stability. Both models effectively captured the dominant visual features of Pokémon types while avoiding excessive overfitting despite the small training set.

A key insight from this study is that deeper or pretrained architectures do not necessarily guarantee better alignment with the target domain. The transfer learning approach showed potential but was limited by the dataset size and class imbalance, which likely caused the model to overemphasize frequent or visually dominant categories (e.g., Normal, Grass, Water). Conversely, the custom CNNs, although simpler, demonstrated a strong ability to generalize under constrained data conditions, confirming that efficiency and proper tuning can often outperform complexity in data-limited environments.

Furthermore, class imbalance and data scarcity remain the primary challenges. With only a few hundred samples and several underrepresented types, the models struggled to distinguish rare categories, indicating the need for a more balanced and extensive dataset to unlock their full predictive potential.

Future Work

Based on the observed results and current limitations, several directions are proposed for future research:

- **Apply K-Fold Cross-Validation** to the *Custom CNN v2* and the *EfficientNetB0 Fine-Tuning Phase 1* models to further validate robustness and mitigate overfitting due to small data splits.
- **Increase the dataset size** by collecting or generating additional labeled Pokémon images, which would allow better representation of underrepresented types and improve generalization.

- **Balance class distributions** through data augmentation, re-sampling, or synthetic image generation techniques (e.g., GANs or diffusion-based models) to reduce the dominance of common types and promote fairer learning across all categories.

These improvements aim to consolidate the findings of this work, enhancing both the accuracy and interpretability of future models while enabling more meaningful transfer learning experiments under better data conditions.

References

Bibliography

- [1] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” *arXiv preprint arXiv:1905.11946*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.11946>
- [2] Scikit-learn, “Cross-validation: Evaluating estimator performance,” n.d. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html
- [3] A. Shervine, “Convolutional Neural Networks Cheatsheet,” Stanford University, n.d. [Online]. Available: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>
- [4] TensorFlow, “Convolutional neural networks tutorial,” n.d. [Online]. Available: <https://www.tensorflow.org/tutorials/images/cnn>
- [5] V. Subbiah, “Pokemon Images and Types [Data set],” Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/vishalsubbiah/pokemon-images-and-types>
- [6] H. Yang, M.-L. Zhang, and Z.-H. Zhou, “Deep learning for multi-label learning: A comprehensive survey,” *arXiv preprint arXiv:2401.16549*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.16549>
- [7] GeeksforGeeks, “Introduction to multilabel classification,” n.d. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/an-introduction-to-multilabel-classification/>